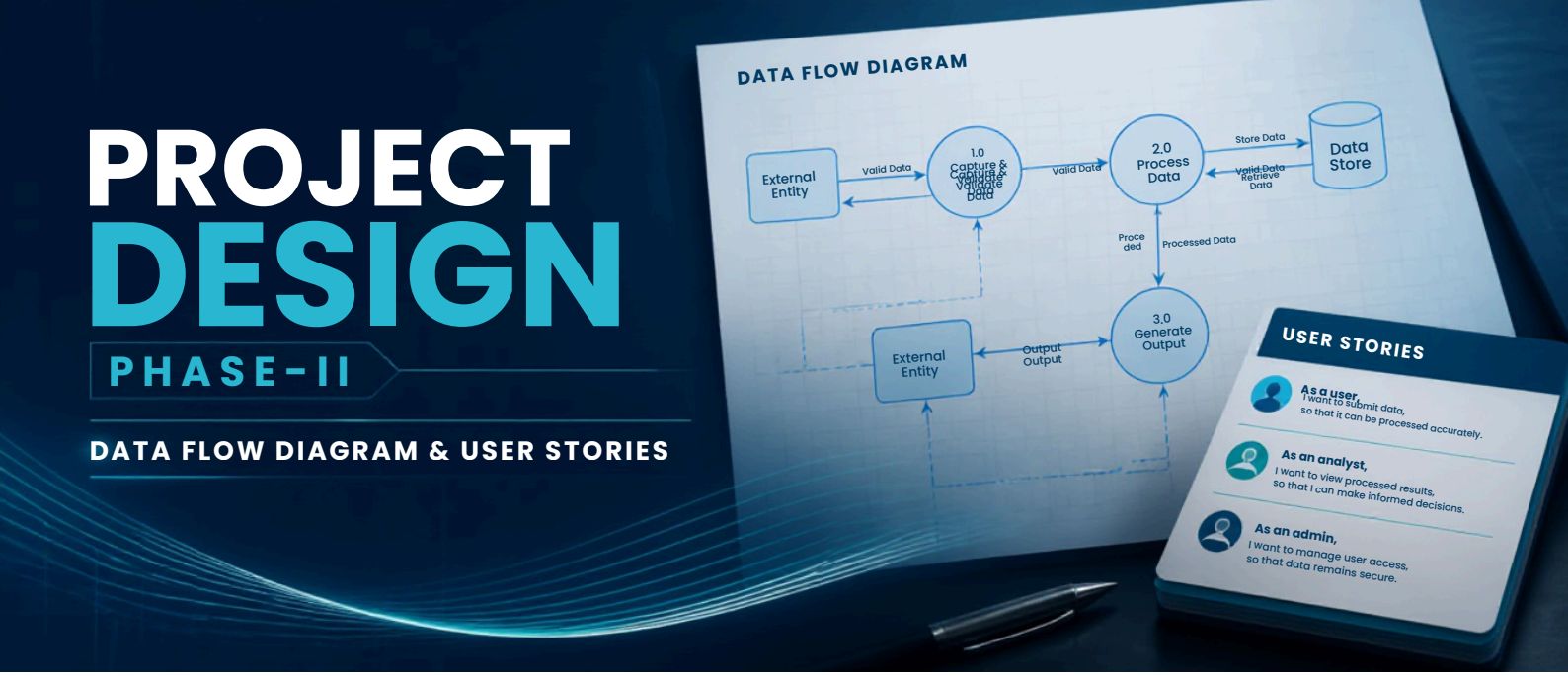


A Comprehensive Measure of Well-Being (HDI Prediction System)

Technical Design Document — Project Design Phase-II

Authors	Paruchuri Venkatesh; Rokkam Guna Sekhar
Tech Stack	Python, Flask, Scikit-Learn, SQLite, SQLAlchemy, Flask-Login, Bootstrap, Jinja2, Pandas, NumPy

Document purpose — Define the Phase-II design for the HDI Prediction System: scope, user stories, data flows, workflows, data entities, requirements, and delivery risks to guide implementation and review.

Project Overview

The HDI Prediction System is a Machine Learning–powered web application that estimates a country's **Human Development Index (HDI)** using key socio-economic indicators. It provides real-time predictions via a Flask web interface, stores prediction history in SQLite, and offers an analytics dashboard for monitoring usage and trends.

Inputs — Country name, life expectancy, mean years of schooling, expected years of schooling, and GNI per capita.

Outputs — Predicted HDI score (0.000–1.000) and UNDP-style development category (Low/Medium/High/Very High).

Phase-II Objectives & Scope

Phase-II focuses on strengthening the platform into a robust, reviewable system design suitable for academic evaluation and dependable operational use. The phase emphasizes clearer workflows, data design, quality attributes, and governance around predictions and records.

Objectives

- Harden end-to-end workflows: authentication, prediction submission, results rendering, history management, and dashboard analytics.
- Formalize data entities for users, inputs, outputs, and history with storage and integrity constraints in SQLite via SQLAlchemy ORM.
- Define functional and non-functional requirements aligned to performance, correctness, security, and maintainability targets.
- Document DFDs and workflow charts to support implementation, testing, and assessment.

In scope

- Web application features: Register/Login, Prediction, Results, Dashboard analytics, Prediction History (view/delete).
- Model-serving behavior for a trained **Linear Regression** model loaded from serialized artifacts (Pickle).
- Security baseline: Flask-Login sessions, password hashing, input validation, ORM-safe queries.

Out of scope (Phase-II)

- Live UNDP/World Bank ingestion, cloud deployment, Docker/CI-CD, REST API, and mobile apps (captured as future enhancements).

Phase-I Summary (Brief)

Phase-I delivered an end-to-end ML prediction platform: data preprocessing and feature scaling, Linear Regression training and evaluation (reported $R^2 \approx 0.9393$; RMSE ≈ 0.0248), model serialization, and Flask-based UI for prediction. It included secure login, SQLite-backed history storage, and a dashboard that visualizes prediction statistics (e.g., total predictions and distribution). Phase-II formalizes and refines the design for clarity, robustness, and extensibility.

Stakeholders & Personas

Stakeholder / Persona	Goals	Primary Needs
Student / Learner	Understand HDI drivers; practice ML deployment concepts	Clear UI, explainable outputs, accessible history
Researcher / Policy Analyst	Rapid estimation for scenario testing and analysis	Repeatable predictions, export-ready history, trend summaries
NGO / Development Planner	Use quick projections to support program planning	Reliable predictions, understandable categories, dashboard overview
Administrator (Research/Admin)	Govern records; oversee system health and usage patterns	Record management, monitoring, auditability, safe deletion rules

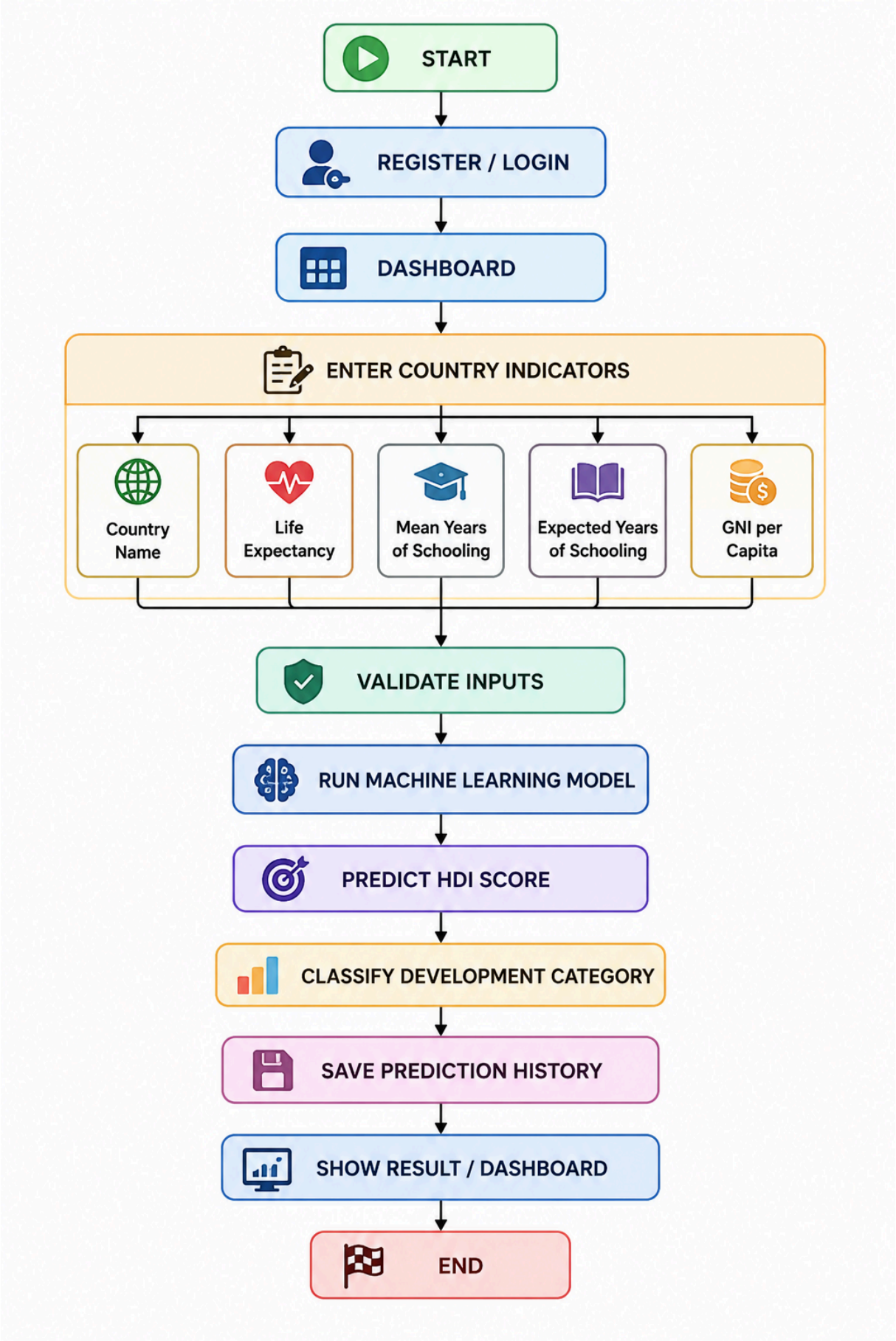
Key decision — Phase-II treats **prediction history** as a first-class feature: every prediction is stored with the user, timestamp, inputs, outputs, and derived category to support analytics and reproducibility.

User Stories (Phase-II)

ID	User story	Acceptance criteria
US-01	As a **new user**, I want to register an account so that I can access predictions securely.	• Registration requires unique email/username and a password. • Password is hashed before storage. • On success, user can log in and reach the dashboard.
US-02	As a **registered user**, I want to log in and stay authenticated so that my history is private.	• Invalid credentials show a clear error without revealing which field is wrong. • Session is established via Flask-Login; protected pages require authentication. • Logout terminates the session and blocks protected pages.
US-03	As a **registered user**, I want to submit development indicators to receive an HDI prediction instantly.	• Input form includes: country, life expectancy, mean years schooling, expected years schooling, GNI per capita. • Server validates numeric ranges and required fields. • System returns HDI score (0–1) and category within an acceptable response time.
US-04	As a **registered user**, I want each prediction saved so I can review results later.	• Each prediction is persisted with timestamp, inputs, outputs, and category. • Records are associated with the authenticated user. • Save occurs only after successful model

		inference and
US-05	As a **registered user**, I want to view and optionally delete my prediction history.	• History lists records in reverse chronological order. • User can view full details of a record (inputs + outputs). • User can delete only their own records; deletion updates dashboard aggregates.
US-06	As a **registered user**, I want a dashboard that summarizes my prediction activity and trends.	• Dashboard shows total predictions, average HDI, and distribution visualization. • New predictions update the dashboard statistics. • Only the user's data is shown (unless admin role is used).
US-07	As an **administrator/research persona**, I want to review overall trends and manage records to ensure data quality.	• Admin can list aggregate statistics across all users (if enabled). • Admin can flag or remove anomalous records with an audit note. • Actions are authorized by role checks and logged.

Data Flow Diagram (DFD)



Flow Charts (Key Workflows)

A) Registration & Login

```
[Start] ↓ [Open Register/Login] ↓ {New user?} ——Yes——> [Register: validate fields] ——> [Hash password] ——> [Create user in SQLite] ——> [Login] │No └————————————————————> [Login: verify credentials] ——> {Valid?} ├—No——> [Show error] ——> [Retry] └—Yes—> [Create session] ——> [Dashboard]
```

B) Prediction Submission

```
[Dashboard] → [Open Predict Page] ↓ [Enter: country, lifeExp, meanSch, expSch, GNI] ↓ [Server-side validation] ↓ {Valid?} ├—No——> [Return form errors] └—Yes—> [Preprocess + scale] → [Load Pickle model] → [Predict HDI] ↓ [Categorize: Low/Medium/High/Very High] ↓ [Persist record in SQLite] ↓ [Return Result View]
```

C) Prediction Result Display

```
[Receive predicted HDI + category] ↓ [Render result card] ↓ [Show: HDI numeric score + category + input recap] ↓ [Links: View History | Back to Predict | Dashboard]
```

D) Prediction History

```
[Authenticated User] ↓ [Open History Page] ↓ [Query SQLite for user's records (desc by timestamp)] ↓ [Render table: inputs + outputs] ↓ {Delete record?} ├—No——> [End] └—Yes—> [Authorize ownership] → [Delete] → [Recompute dashboard aggregates]
```

E) Dashboard Updates

```
[New prediction saved OR record deleted] ↓ [Analytics module queries aggregates] ↓ [Compute: total predictions, avg HDI, distribution] ↓ [Render charts + recent activity]
```

Data Entities & Storage Notes

Entity	Key fields (illustrative)	Notes
Users	id, name/username, email, password_hash, role, created_at, last_login_at	Passwords stored hashed (Werkzeug). Role supports admin/research capabilities.
PredictionInputs	id, user_id, country, life_expectancy, mean_schooling, expected_schooling, gni_per_capita, created_at	Numeric fields stored as float; validate ranges and required values.
PredictionOutputs	id, input_id, hdi_score, category, model_name, model_version, inference_ms	Store model identifiers for reproducibility; category derived from score thresholds.
History (View)	join(User, Inputs, Outputs) ordered by created_at desc	Rendered as a table; deletion removes associated input/output records (transactional).

Storage approach — SQLite is used for local persistence; SQLAlchemy ORM ensures parameterized queries and consistent migrations (where applicable). Prediction records are write-once per successful inference to preserve auditability.

Non-Functional Requirements

- **Security** — Password hashing, session protection, CSRF protections where applicable, least-privilege authorization for admin actions, and input validation on both client and server.
- **Performance** — Prediction response should be near real-time; model inference should complete within milliseconds under normal load, with overall request latency within a few seconds including rendering and storage.
- **Correctness** — Predicted HDI constrained to [0, 1] (clamped if required); category thresholds applied deterministically; history reflects persisted truth.
- **Reliability** — Transactional writes for inputs+outputs; graceful error handling for model load failures and DB errors.
- **Maintainability** — Modular Flask blueprint structure (auth/predict/dashboard/history), configuration-driven thresholds, and reproducible model artifact management.

Risks & Assumptions

Assumptions

- The serialized model artifact is compatible with the runtime (library versions aligned) and available at application startup.
- Input indicators follow the same meaning and scaling as the training dataset used in Phase-I.

Risks

- **Data drift** — Changes in real-world socio-economic patterns may reduce prediction quality over time.
- **Model/version mismatch** — Pickle incompatibilities or dependency differences can break inference.
- **Privacy** — Prediction history may contain sensitive analysis; ensure strict per-user isolation and secure session handling.
- **Misinterpretation** — Users may treat predictions as official HDI; mitigate with clear labeling and references to UNDP definitions.

Future Enhancements

- Advanced models (e.g., XGBoost/LSTM) and periodic retraining pipelines.
- Explainable AI (SHAP) to show feature contributions and improve transparency.
- Country comparison dashboards and interactive world map visualization.
- Live UN/World Bank data integration, automated dataset refresh, and validation checks.
- REST API, PDF/Excel export, and report generation for research workflows.
- Containerization (Docker) and cloud deployment (AWS/Azure) with CI/CD and monitoring.
- Multi-language UI and mobile-friendly enhancements beyond responsive web design.

Conclusion

This Phase-II technical design document formalizes the HDI Prediction System's architecture and implementation plan across workflows, data entities, and requirements. The proposed design builds on Phase-I's ML deployment (Flask + Scikit-Learn) and strengthens the platform's security, data integrity, and analytics capabilities using SQLite and SQLAlchemy. The resulting system provides a clear, academically reviewable blueprint for delivering a reliable, user-centered HDI prediction and trend-monitoring platform.